OFF-DYNAMICS REINFORCEMENT LEARNING TRAINING FOR TRANSFER WITH DOMAIN CLASSIFIERS

Benjamin Eysenbach, Swapnil Asawa, Shreyas Chaudhari, Sergey Levine, Ruslan Salakhutdinov

Name: Hongming Zhang
Date: 2022.08.05

Introduction

中国科学院
自动化研究所
INSTITUTE OF AUTOMATION
CHINESE ACADEMY OF SCIENCES

- ICLR 2021

DARC: Domain Adaptation with Rewards from Classifiers, an algorithm for domain adaptation to dynamics changes in reinforcement learning (RL), based on the idea of compensating for differences in dynamics by modifying the reward function.

The agent is penalized for transitions that would indicate that the agent is interacting with the source domain, rather than the target domain, by learning auxiliary classifiers.

Motivation

Many domains where we would like to learn policies are not amenable to such trial-and-error learning, such as autonomous driving. We may have access to a different but structurally similar source domain. Experience in the source domain is much cheaper to collect.

We aim to use a safer source domain, such as a simulator, to train a policy that can then function effectively in a target domain, by compensating for the difference in dynamics by modifying the reward function.

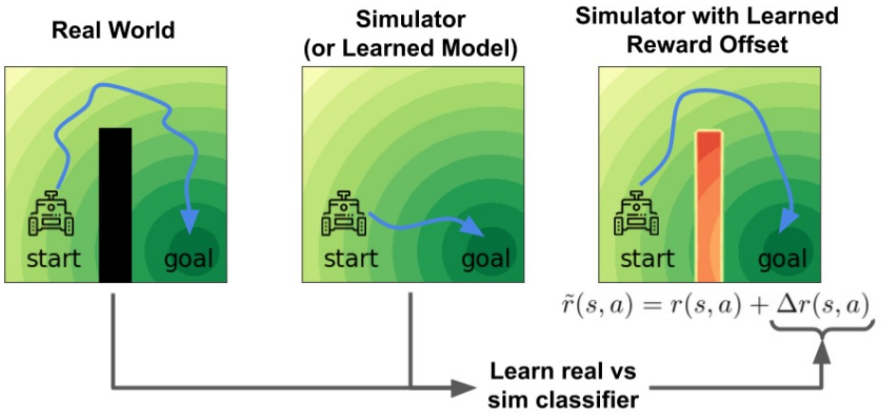


Figure 1: Our method acquires a policy for the target domain by practicing in the source domain using a (learned) modified reward function.

Problem Setting

- Two MDPs:

$\mathcal{M}_{\text{source}}$: the source domain. A practice facility, simulator, or learned model of the target domain)

$\mathcal{M}_{\text{target}}$: the target domain. The problem we want to solve.

- Prerequisites

Assume the two domains have the same state space $\mathcal{S}$, action space $\mathcal{A}$, reward function r, and initially state distribution $p_1(s_1)$.
The only difference is the dynamics, $p_{\text{source}}(s_{t+1}|s_t, a_t)$ and $p_{\text{target}}(s_{t+1}|s_t, a_t)$

- Objective

Learn a policy π that maximizes the expected discounted sum of rewards on $\mathcal{M}_{\text{target}}$.

Problem Setting

Definition 1. *Domain Adaptation for RL is the problem of using interactions in the source MDP $\mathcal{M}_{source}$ together with a small number of interactions in the target MDP $\mathcal{M}_{target}$ to acquire a policy that achieves high reward in the target MDP, $\mathcal{M}_{target}$.*

- Extra Assumption

The agent's experience in the source domain should look similar to its experience in the target domain.

Assume every transition with non-zero probability in the target domain will have non-zero probability in the source domain:

$$p_{\text{target}}(s_{t+1}|s_t, a_t) > 0 \Longrightarrow p_{\text{source}}(s_{t+1}|s_t, a_t) > 0 \quad \text{for all } s_t, s_{t+1} \in \mathcal{S}, a_t \in \mathcal{A}$$

Method

Consider the probabilistic inference interpretation of RL. Let's treat the reward function as defining a desired distribution over trajectories, the optimal policy is proportional to their exponentiated reward.

Define $p(\tau)$ as the desired distribution over trajectories in the target domain:

$$p(\tau) \propto p_1(s_1)(\prod_t p_{\text{target}}(s_{t+1}|s_t, a_t))\exp(\sum_t r(s_t, a_t))$$

and $q(\tau)$ as our agent's distribution over trajectories in the source domain:

$$q(\tau) = p_1(s_1) \prod_t p_{\text{source}}(s_{t+1}|s_t, a_t)\, \pi_\theta(a_t|s_t)$$

Method

Our aim is to learn a policy whose behavior in the source domain both receives high reward and has high likelihood under the target domain dynamics.

We codify this objective by minimizing the reverse KL divergence between these two distributions:

$$\min_{\pi(a|s)} D_{KL}(q\|p) = -\mathbb{E}_{p_{\text{source}}}[\sum_t r(s_t, a_t) + \mathcal{H}_\pi[a_t|s_t] + \Delta r(s_{t+1}, s_t, a_t)] + c,$$

where $\Delta r(s_{t+1}, s_t, a_t) \triangleq \log p(s_{t+1}|s_t, a_t) - \log q(s_{t+1}|s_t, a_t)$

Method

$$\min_{\pi(a|s)} D_{KL}(q\|p) = -\mathbb{E}_{p_{\text{source}}}[\sum_t r(s_t,a_t) + \mathcal{H}_\pi[a_t|s_t] + \Delta r(s_{t+1},s_t,a_t)] + c,$$

where $\Delta r(s_{t+1},s_t,a_t) \triangleq \log p(s_{t+1}|s_t,a_t) - \log q(s_{t+1}|s_t,a_t)$

$$p(\tau) \propto p_1(s_1)(\prod_t p_{\text{target}}(s_{t+1}|s_t,a_t))\exp(\sum_t r(s_t,a_t))$$

$$q(\tau) = p_1(s_1)\prod_t p_{\text{source}}(s_{t+1}|s_t,a_t)\,\pi_\theta(a_t|s_t)$$

$$\begin{aligned}
D_{KL}(q\|p) &= \mathbb{E}_q[\log q - \log p] + c \\
&= \mathbb{E}_q[\log p_1(s_1) + \textstyle\sum_t \log p_{\text{source}}(s_{t+1}|s_t,a_t) + \sum_t \log \pi_\theta(a_t|s_t) \\
&\qquad -\log p_1(s_1) - \textstyle\sum_t \log p_{\text{target}}(s_{t+1}|s_t,a_t) - \sum_t r(s_t,a_t)] + c \\
&= -\mathbb{E}_q\big[\textstyle\sum_t r(s_t,a_t) - \sum_t \log \pi_\theta(a_t|s_t) + \sum_t \log p_{\text{target}}(s_{t+1}|s_t,a_t) - \sum_t \log p_{\text{source}}(s_{t+1}|s_t,a_t)\big] + c \\
&= -\mathbb{E}_q[\textstyle\sum_t (r(s_t,a_t) - \log \pi_\theta(a_t|s_t) + \Delta r(s_{t+1},s_t,a_t))]
\end{aligned}$$

$$\mathbb{E}_q = \mathbb{E}_{\substack{s_1 \sim p_1(s_1) \\ s_{t+1} \sim p_{\text{source}}(s_{t+1}|s_t,a_t) \\ a_t \sim \pi_\theta(a_t|s_t)}}$$

Method

$$\min_{\pi(a|s)} D_{KL}(q\|p) = -\mathbb{E}_{p_{\text{source}}}[\sum_t r(s_t, a_t) + \mathcal{H}_\pi[a_t|s_t] + \Delta r(s_{t+1}, s_t, a_t)] + c,$$

where $\Delta r(s_{t+1}, s_t, a_t) \triangleq \log p(s_{t+1}|s_t, a_t) - \log q(s_{t+1}|s_t, a_t)$

This objective suggests a corrective term Δr that should be added to the reward function to account for the discrepancy between the source and target dynamics.

When $\Delta r = 0$, the source and target dynamics are equal and we recover maximum entropy RL.

When $\Delta r < 0$, the transitions are likely in the source but are unlikely in the target domain. The agent is penalized for "exploiting" inaccuracies or discrepancies in the source domain.

Method

To estimate Δr, we will use a pair of (learned) binary classifiers, which will infer whether transitions came from the source or target domain.

$$\begin{aligned}\Delta r(s_{t+1}, s_t, a_t) &\triangleq \log p(s_{t+1}|s_t, a_t) - \log q(s_{t+1}|s_t, a_t) \\ &= \log p_{\text{target}}(s_{t+1}|s_t, a_t) - \log p_{\text{source}}(s_{t+1}|s_t, a_t) \\ &= \log p(\text{target}|\text{s}_\text{t}, a_t, s_{t+1}) - \log p(\text{target}|\text{s}_\text{t}, a_t) \\ &\quad -\log p(\text{source}|\text{s}_\text{t}, a_t, s_{t+1}) + \log p(\text{source}|\text{s}_\text{t}, a_t)\end{aligned}$$

Via Bayes' rule:

$$\begin{aligned}p_{\text{target}}(s_{t+1}|s_t, a_t) &= p(s_{t+1}|s_t, a_t, \text{target}) \\ &= \frac{p(s_t, a_t, s_{t+1}, \text{target})}{p(s_t, a_t, \text{target})} \\ &= \frac{p(\text{target}|s_t, a_t, s_{t+1})}{p(\text{target}|s_t, a_t)} \cdot \frac{p(s_t, a_t, s_{t+1})}{p(s_t, a_t)}\end{aligned}$$

Method

To estimate Δr, we will use a pair of (learned) binary classifiers, which will infer whether transitions came from the source or target domain.

$$\begin{aligned}\Delta r(s_{t+1}, s_t, a_t) &\triangleq \log p(s_{t+1}|s_t, a_t) - \log q(s_{t+1}|s_t, a_t) \\ &= \log p_{\text{target}}(s_{t+1}|s_t, a_t) - \log p_{\text{source}}(s_{t+1}|s_t, a_t) \\ &= \log p(\text{target}|\text{s}_\text{t}, a_t, s_{t+1}) - \log p(\text{target}|\text{s}_\text{t}, a_t) \\ &\quad -\log p(\text{source}|\text{s}_\text{t}, a_t, s_{t+1}) + \log p(\text{source}|\text{s}_\text{t}, a_t)\end{aligned}$$

- Intuitively, Δr answers the following question: for the task of predicting whether a transition came from the source or target domain, how much better can you perform after observing s_{t+1}.
- While our method is not solving an imitation learning problem (we do not assume access to any expert experience), our method can be interpreted as learning a policy such that the dynamics observed by that policy in the source domain imitate the dynamics of the target domain.

Algorithm

Our algorithm modifies an existing MaxEnt RL algorithm (SAC) to additionally learn two classifiers, $q_{\theta_{SAS}}(\text{target}|s_t, a_t, s_{t+1})$ and $q_{\theta_{SA}}(\text{target}|s_t, a_t)$ by minimising the standard cross-entropy loss.

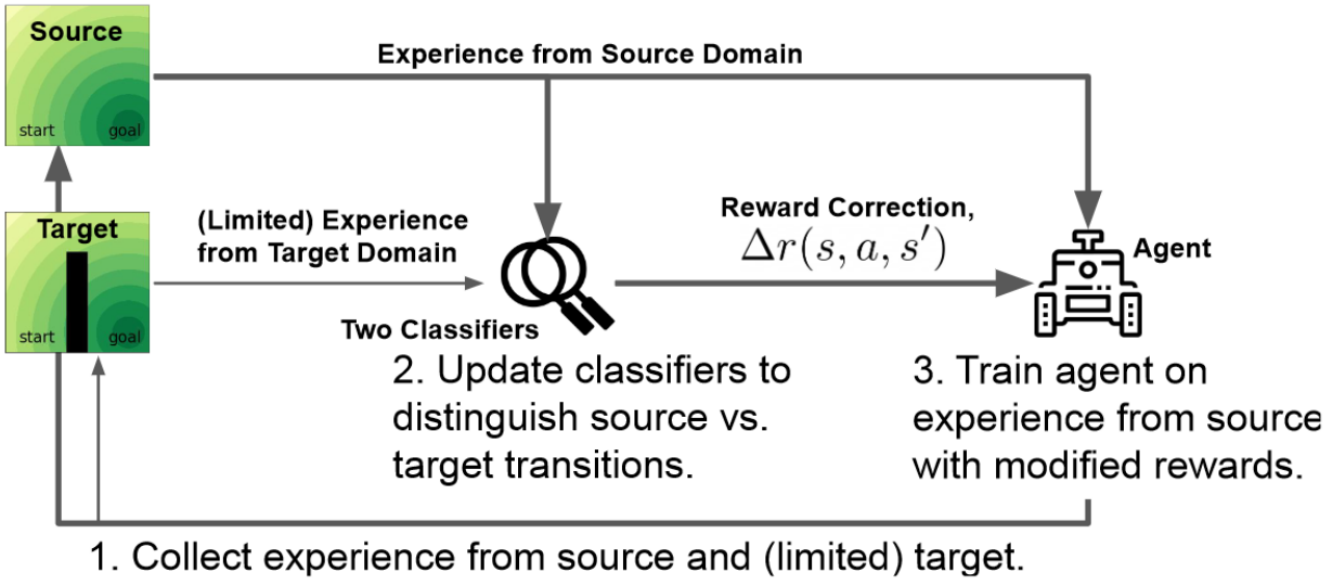


Figure 2: Block diagram of DARC (Alg. 1)

Algorithm

中国科学院 自动化研究所 INSTITUTE OF AUTOMATION CHINESE ACADEMY OF SCIENCES

Our algorithm modifies an existing MaxEnt RL algorithm (SAC) to additionally learn two classifiers, $q_{\theta_{SAS}}(\text{target}|s_t, a_t, s_{t+1})$ and $q_{\theta_{SA}}(\text{target}|s_t, a_t)$ by minimising the standard cross-entropy loss.

Algorithm 1 Domain Adaptation with Rewards from Classifiers [DARC]

Input: source MDP $\mathcal{M}_{\text{source}}$ and target $\mathcal{M}_{\text{target}}$; ratio r of experience from source vs. target.
Initialize: replay buffers for source and target transitions, $\mathcal{D}_{\text{source}}, \mathcal{D}_{\text{target}}$; policy π; parameters $\theta = (\theta_{\text{SAS}}, \theta_{\text{SA}})$ for classifiers $q_{\theta_{\text{SAS}}}(\text{target} \mid s_t, a_t, s_{t+1})$ and $q_{\theta_{\text{SAS}}}(\text{target} \mid s_t, a_t)$.
for $t = 1, \cdots,$ num iterations **do**
  $\mathcal{D}_{\text{source}} \leftarrow \mathcal{D}_{\text{source}} \cup \text{ROLLOUT}(\pi, \mathcal{M}_{\text{source}})$ ▷ Collect source data.
  if $t \mod r = 0$ **then** ▷ Periodically, collect target data.
    $\mathcal{D}_{\text{target}} \leftarrow \mathcal{D}_{\text{target}} \cup \text{ROLLOUT}(\pi, \mathcal{M}_{\text{target}})$
  $\theta \leftarrow \theta - \eta \nabla_\theta \ell(\theta)$ ▷ Update both classifiers.
  $\tilde{r}(s_t, a_t, s_{t+1}) \leftarrow r(s_t, a_t) + \Delta r(s_t, a_t, s_{t+1})$ ▷ Δr is computed with Eq. 3.
  $\pi \leftarrow \text{MAXENT RL}(\pi, \mathcal{D}_{\text{source}}, \tilde{r})$
return π

Experiments

Toy example.

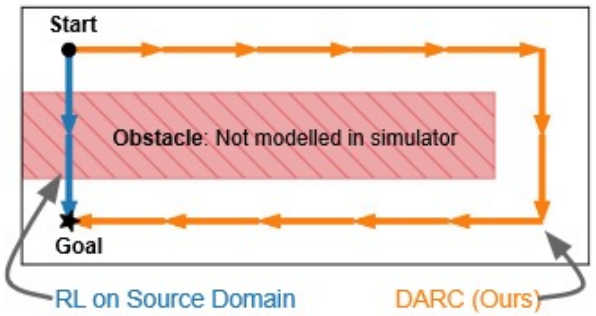


Figure 3: **Tabular example of off-dynamics RL**

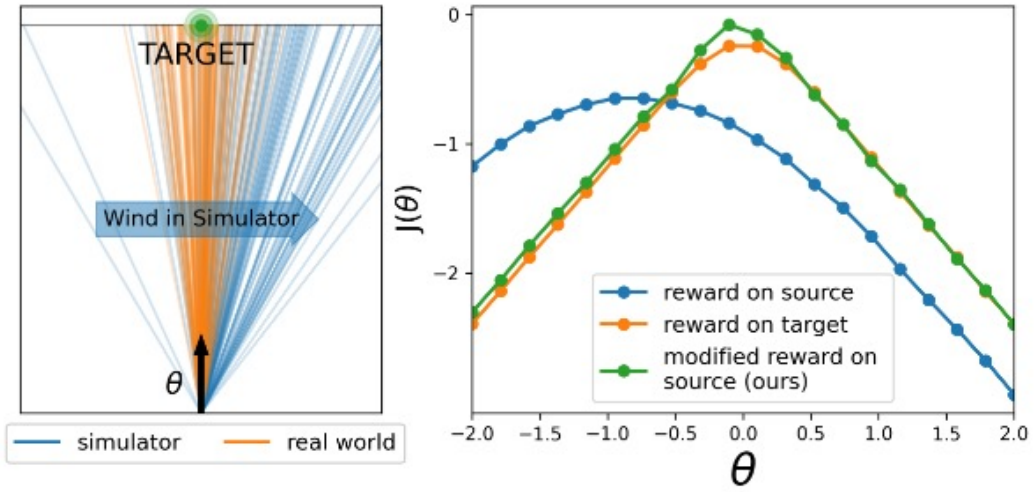


Figure 4: **Visualizing the modified reward**

Experiments

Overall performance on continuous control tasks.

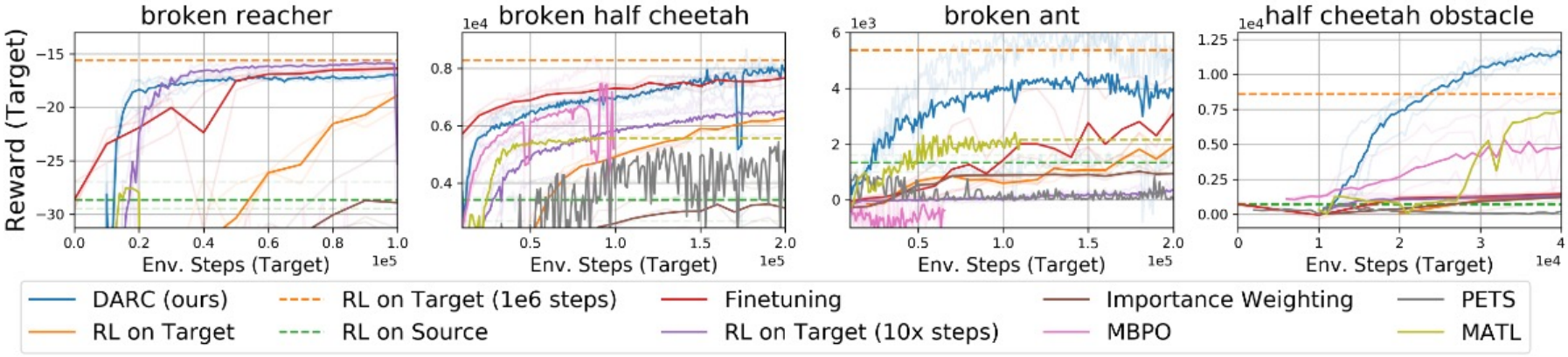
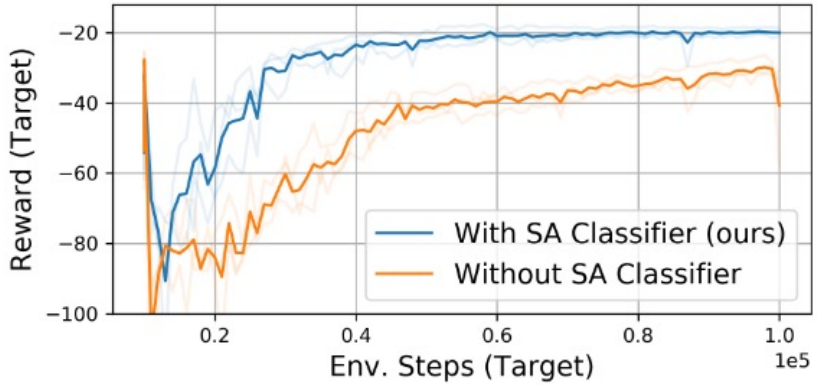


Figure 6: **DARC compensates for crippled robots and obstacles**: We apply DARC to four continuous control tasks: three tasks (broken reacher, half cheetah, and ant) which are crippled in the target domain but not the source domain, and one task (half cheetah obstacle) where the source domain omits the obstacle from the target domain. Note that naïvely ignoring the shift in dynamics (green dashed line) performs quite poorly, while directly learning on the crippled robot requires an order of magnitude more experience than our method.

Experiments

Ablation study.



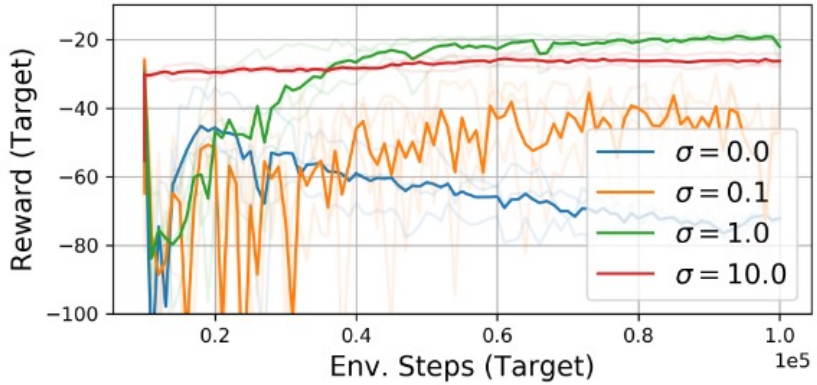


Figure 7: **Ablation experiments** *(Left)* DARC performs worse when only one classifier is used. *(Right)* Using input noise to regularize the classifiers boosts performance. Both plots show results for broken reacher; see Appendix D for results on all environments.

Conclusion

中国科学院
自动化研究所
INSTITUTE OF AUTOMATION
CHINESE ACADEMY OF SCIENCES

Prior work has studied the domain adaptation of observations in RL. This paper proposed DARC algorithm for domain adaptation to changing dynamics in RL.

DARC compensates for differences in dynamics via the reward function, which is a novel perspective. This modified reward function is simple to estimate by learning auxiliary classifiers and does not require learning an explicit model of the dynamics.

Experiments show that DARC can leverage the source domain to learn policies that will work well in the target domain, despite observing only a handful of transitions from the target domain.

中国科学院
自动化研究所
INSTITUTE OF AUTOMATION
CHINESE ACADEMY OF SCIENCES

THANKS